

프로젝트 보고서

1. 개요

이번 프로젝트는 암호화된 네트워크 트래픽을 분석하여 앱을 식별하는 모델을 개발하는 것을 목표로 한다. Internet Service Provider 등은 전통적으로 패킷 페이로드 분석 및 SN1 기반 앱 식별 기술을 통해 QoS 관리 및 특정 앱 차단 등을 수행했다. 하지만 기술의 발달로 인하여 대부분의 네트워크 트래픽은 TLS, QUIC, VNP와 같은 보안 프로토콜을 통해 암호화되고 있으며, 이러한 환경에서는 기존의 패킷 페이로드 기반 앱 식별 기술이 더 이상 활용될 수 없다. 패킷 내부 내용을 직접 확인할 수 없기 때문에 패킷 길이, 방향, 도착 시간과 같은 메타데이터를 기반으로 통계적 특징을 추출하여 분류하는 방식으로 접근해야 한다.

2. 데이터 분석

Train 데이터셋은 총 29552개의 flow로 구성되어 있으며, 각 flow는 단일 앱 라벨을 가진다. 라벨의 수도 77개로 꽤 많은 편이라 데이터셋 내에는 다양한 애플리케이션 패턴이 존재한다. 라벨별 분포를 분석한 결과 상위 라벨은 거의 400개에 달하는 flow를 보유하고 있었지만 하위 라벨들은 100개 이하인 경우도 다수 존재했다. 이러한 분포는 데이터가 완전히 균등하지 않으며 일부 클래스에 모델이 과도하게 맞춰질(overfit) 가능성이 있음을 의미한다. 따라서 모델 안에서 `class_weight='balanced'` 옵션을 활용하여 불균형 문제를 일부 해결하고자 하였다. 실제로 모델 학습 과정에서 균형 가중치를 적용하는 것이 F1-score 향상에 도움이 되었다.

각 flow의 구성 요소인 `length_sequence`, `direction_sequence`, `time_sequence`는 모두 리스트 형태의 시퀀스 데이터로 제공된다. `length_sequence`는 패킷 길이의 나열로 이루어져 있으며 최소 길이는 10, 최대 길이는 131182로 나타났다. 이는 앱 종류나 네트워크 상황에 따라 패킷 수가 상당히 많이 차이 날 수 있음을 보여준다. 예를 들면 일부 스트리밍 서비스나 대용량 다운로드 기반 앱은 길이가 매우 긴 flow를 생성하는 경향이 있다. 반면 간단한 요청/응답 구조의 앱은 매우 짧은 흐름만을 생성한다. `direction_sequence`는 패킷의 송신/수신 여부를 나타내며 1(다운로드), -1(업로드)로 구성된다. `direction_sequence`의 길이 분포도 `length_sequence`와 동일하게 최소 10에서 최대 131182까지 분포하고 있었으며 대부분의 flow에서 downstream 패킷의 비중이 높았다. 이는 오늘날 앱들이 서버에서 클라이언트로 데이터를 전달하는 방식이 많다는 것을 보여주는 것 같다.

`time_sequence`는 패킷 도착 시간을 나타내는 시퀀스로 이를 기반으로 Inter-Arrival Time

을 계산할 수 있다. 시간 기반 데이터는 네트워크 지연, 트래픽 burst 패턴, 사용자 입력 여부 등 다양한 요인을 반영하는 중요한 정보이다. 실제로 Inter-Arrival Time을 계산해보면 많은 flow에서는 패킷들이 거의 쉬지 않고 빠르게 연달아 도착하는 경우가 많았다. 반대로, 길이가 긴 flow 중 일부는 중간중간에 패킷이 오지 않는 빈 구간이 생기기도 했다. 이 차이는 앱이 어떤 방식으로 동작하는지와 직접적으로 관련되어 있으며 앱을 구분하는 데 중요한 단서가 된다.

3. 피처

시퀀스 기반 데이터는 정보가 풍부하지만 그 길이의 편차가 극단적으로 크다는 점이 문제였다. 이 데이터 그대로 모델 학습을 시키기에는 비효율적일 뿐만 아니라 딥러닝 기반 모델을 사용한다 하더라도 GPU 메모리가 부족할 가능성이 높다. 또한 길이가 오차 범위를 넘어서게 되면 패딩이 지나치게 많아져 오히려 의미 있는 패턴을 왜곡할 가능성이 있다. 이러한 이유로 max_len=40을 기준으로 시퀀스를 잘라내거나 패딩을 적용하였다. 패딩 값은 3000으로 설정했는데 이는 패딩과 실제 데이터의 값을 구분하기 쉽도록 설계된 방식이다. 실제 패킷 길이의 분포가 보통 1500 이하인 점을 고려하면 3000은 데이터가 없음을 명확히 표현할 수 있는 값이라고 볼 수 있다.

시퀀스를 고정 길이로 변환한 뒤에는 flow의 특징을 더욱 풍부하게 표현하기 위해 여러 통계 기반 피처를 추가하였다. length_sequence로부터는 평균, 중앙값, 최댓값, 표준편차, 1분위수(Q1)와 3분위수(Q3)를 계산하였다. 이 피처들은 앱 종류 간의 패킷 길이 분포 차이를 반영하는 데 매우 효과적이다. direction_sequence에서는 forward 패킷 개수, backward 패킷 개수, 양 방향의 비율, 그리고 direction_change 횟수를 계산하였다. direction_change는 특히 상호작용 중심 앱에서 높게 나타나는 경향이 있으며, 단방향 전송이 많은 앱과 구분하는 데 도움을 준다. time_sequence 기반으로 계산한 IAT는 중요한 피처 중 하나다. Inter-Arrival Time의 평균, 최소, 최대, 표준편차 등을 통해 트래픽의 시간적 구조를 정량적으로 표현할 수 있었다. 시간적 특징은 단순한 패킷 크기 정보만으로는 드러나지 않는 앱 특성을 반영하기 때문에 모델이 더 풍부한 정보를 학습할 수 있도록 돕는다. 이렇게 뽑아낸 다양한 피처들은 시퀀스 기반 정보와 통계 기반 정보를 결합하여 flow의 전반적인 구조를 포괄적으로 설명하는 역할을 했다.

4. 모델

데이터 학습에 사용된 모델은 LightGBM이다. LightGBM은 Gradient Boosting Decision Tree(GBDT) 계열의 모델로 다양한 형태의 피처들이 혼합된 데이터에서 높은 성능을 보

이며 학습 속도가 빠르고 메모리 효율이 좋다는 장점이 있다. 또한 과적합을 방지하기 위한 다양한 하이퍼파라미터를 제공하여 모델을 유연하게 조정할 수 있다. 중간 수준의 적절한 값인 `n_estimators=500`으로 모델이 500개의 트리를 순차적으로 학습하도록 하였다. `num_leaves=16`은 각 트리가 분기할 수 있는 최대 리프 노드 수를 제한하는 하이퍼파라미터인데, 이 값을 크게 했더니 과적합이 발생하여 오히려 성능이 떨어지는 결과가 나타났다. 그래서 데이터의 크기와 피처 구성에 적합한 값인 16을 사용해 모델 복잡도를 제어하였다. `min_child_samples=50`은 한 리프 노드가 갖출 최소 데이터 수를 의미한다. 리프 노드에 너무 적은 샘플만 존재하게 되면 모델이 데이터의 작은 변동에 민감해지고 과적합이 발생할 수 있기 때문에 충분한 샘플 수를 강제하는 방식으로 모델의 안정성을 높였다. `learning_rate=0.05`로 모델이 한 번에 너무 빠르게 학습하지 않도록 조절했다. 추가적으로 `subsample`과 `colsample_bytree`는 각각 데이터 샘플과 피처 일부만을 무작위로 선택하여 각 트리를 학습하게 하는 설정이다. 이러한 무작위성은 개별 트리가 서로 다른 관점을 학습하도록 유도하며 결과적으로 모델의 일반화 성능을 향상시킬 수 있다. 마지막으로 `class_weight='balanced'`를 적용하여 데이터 라벨의 불균형 문제를 보정하였다. 이렇게 하면 클래스 빈도에 따라 가중치를 자동으로 조정해주기 때문에 희귀 클래스도 충분히 반영될 수 있다.

모델 평가에는 train-valid split 방식이 사용되었으며 평가 지표로는 accuracy와 macro F1-score를 활용하였다. accuracy는 0.9049로 나타났으며 macro F1-score 역시 accuracy와 비슷한 0.9051의 높은 수치를 기록하였다. 특히 macro F1-score는 클래스 간 불균형이 존재하는 상황에서도 모델이 모든 라벨을 얼마나 균형 있게 예측하는지를 나타내는 지표이기 때문에 본 모델이 다양한 라벨에 대해 안정적인 예측 성능을 보였다는 것을 의미한다. 이는 본 프로젝트에서 추가로 뽑아낸 피처들이 실제로 모델 성능 향상에 효과적이었음을 확인시켜준다.

5. 결론

이번 프로젝트를 수행하면서 패킷 페이로드 없이 메타데이터만으로 특정 앱을 높은 정확도로 식별할 수 있다는 점을 직접 경험할 수 있었다. 이는 실제 네트워크 보안에서 매우 중요한 기술이며 암호화 환경에서도 트래픽 분석과 분류가 가능하다는 것을 보여준다. 또한 시퀀스 길이에 극단적인 차이가 존재하는 데이터에서는 단순히 모델 복잡도를 높이는 것보다는 적절한 통계적 요약과 피처 선택이 훨씬 효과적이라는 점도 확인할 수 있었다.